
Archivos

Introducción

- *En el lenguaje "C" todos los dispositivos del computador son considerados como archivos.*
- *Esto es muy similar a la forma de trabajo del Sistema Operativo Unix.*
- *Debemos recordar que el lenguaje “C” no contiene sentencia alguna de manejo de entrada o salida.*

Introducción

- *La biblioteca estándar es la encargada de suministrar las funciones de entrada y salida.*
- *Esto hace al "C" extremadamente flexible y potente.*
- *Podemos transferir los datos desde una entrada o hacia una salida en forma de texto o con el formato de la representación binaria interna.*

Introducción

- *El estándar ANSI redefinió completamente el manejo de la entrada/salida.*
- *Dejó vigente solamente lo que se denomina **sistema de archivos con buffer**, descartando el sistema de archivos sin buffer utilizado por el Unix.*
- *Es posible encontrar el sistema sin buffer en programas Unix antiguos. No se debería utilizar para nuevos programas.*

Corrientes (Secuencias)

- *El tratamiento unificado de la comunicación que realiza el "C" con el exterior se basa en una abstracción denominada: **corrientes**.*
- *El sistema se encarga de conectar las corrientes con controladoras de disco, controladoras de cinta magnética, impresoras, terminales, etc.*
- *Se conecta **corrrientes** a **archivos**.*

Corrientes (Secuencias)

- *Todas las corrientes son iguales.*
- *Los dispositivos (archivos) no.*
- *Por ejemplo es posible volver hacia atrás en un acceso a disco, pero no en una salida a impresora.*
- *Es posible tener acceso aleatorio en un archivo de disco, pero no en uno de cinta magnética.*

Tipos de Corrientes

- *El trabajo con archivos, implica la utilización de dos clases de corrientes:*
  - ***Corrientes de texto***
  - ***Corrientes binarias***
- *Las corrientes de texto son una secuencia de caracteres terminada por un fin de línea.*
- *Los fines de línea difieren en los sistemas operativos.*

Corrientes de Texto

- *Para el sistema operativo Unix, el fin de línea está representado por el carácter ASCII 10.*
- *En DOS/Windows el fin de línea se representa por la secuencia ASCII 13 10.*
- *Estas dos formas representan la misma acción "**\n**" y su efecto es que el cursor se desplace al principio de la línea siguiente.*

Corrientes Binarias

- *Una corriente binaria es una secuencia de caracteres sin organización alguna.*
- *Podríamos decir que toda secuencia que no es de texto es binaria.*
- *En estas secuencias se representan imágenes, ejecutables, sonido, almace-namiento en el formato interno, etc.*
- *La corriente binaria solo es interpretada por una aplicación.*

Corrientes

- *Esta forma de tratamiento no es privativa del lenguaje "C".*
- *En la Internet cuando transferimos archivos mediante el protocolo adecuado:* ***ftp****, tenemos la misma diferenciación.*
- *Hay que ser muy cuidadoso de no confundir una corriente de texto con una corriente binaria, el resultado sería desastroso la mayoría de las veces.*

Archivos

- *Para comunicar una corriente con un archivo (dispositivo) debemos asociarla a él.*
- *Esto se hace utilizando una estructura denominada: **FILE**, de la biblioteca estándar, reflejada en el archivo de cabecera stdio.h*
- *Declaramos un puntero a esta estructura que representará nuestra corriente.*

Estructura FILE

```
typedef struct
{
        unsigned char  *curp;       /* Current active pointer     */
        unsigned char  *buffer;     /* Data transfer buffer       */
        int             level;      /* fill/empty level of buffer */
        int             bsize;      /* Buffer size                */
        unsigned short  istemp;     /* Temporary file indicator   */
        unsigned short  flags;      /* File status flags          */
        wchar_t         hold;       /* Ungetc char if no buffer   */
        char            fd;         /* File descriptor            */
        unsigned char   token;      /* Used for validity checking */
}       FILE;                       /* This is the FILE object    */
```

Archivos

- *Dentro de FILE están todos los elementos necesarios para controlar la corriente de comunicación.*
- *La declaración se realiza así:*

```
FILE *fp;
FILE *in, *out;
FILE *entrada, *salida;
FILE *ent1, *ent2, *transf;
```

Archivos

- *La conexión de una corriente con un archivo la realizamos así:*

```
FILE *fp;
fp = fopen( "archivo.ext", "r" );
if( fp == NULL ){
  printf( "Error de apertura\n" );
  exit( 1 );
}
```

- *¡ Este control **siempre** debe realizarse !*

fopen()

- *fopen() es una función que tiene el formato:*

```
FILE *fopen( char *nombre_archivo,
             char *modo );
```

- *Esto relaciona una corriente con un dispositivo (archivo).*
- *Tiene dos parámetros: el **nombre del archivo** y el **modo de apertura**.*

Modos de Apertura

- *Los modos de apertura de una corriente de texto son:*

```
"r"    equivale a    "rt"
"w"    equivale a    "wt"
"a"    equivale a    "at"
"r+"   equivale a    "r+t"
"w+"   equivale a    "w+t"
"a+"   equivale a    "a+t"
```

Apertura de corriente de texto

Modo:

```
"r"  : lectura
"w"  : creación para escritura
"a"  : escritura al final
"r+" : lectura y escritura
"w+" : creación para lectura/escrit.
"a+" : lectura/escritura al final
```

Si hemos abierto en un modo, podemos cerrar el archivo y abrir en otro modo.

Apertura de corriente binaria

- *Los modos de apertura de una corriente binaria son:*

```
"rb"   lectura
"wb"   creación para escritura
"ab"   escritura al final
"r+b"  lectura y escritura
"w+b"  creación para lect./escrit.
"a+b"  lectura/escritura al final
```

Cierre de un Archivo

- *Independientemente del modo en que se abrió un archivo, se lo cierra mediante:*

```
fclose( nombre );
```

- *Esto descarga el buffer, haciendo que efectivamente se trasladen los contenidos.*
- *Es importante cerrar los archivos.*

Cierre de un Archivo

- *Si al terminar de ejecutarse un programa no se cerraron los archivos mediante la sentencia específica, el sistema operativo se encarga de cerrarlos.*
- *No deberíamos descansar la seguridad de nuestras operaciones en lo anterior.*
- *La forma general es:*

```
int fclose( FILE *fp );
```

Cierre de un Archivo

- *Una operación de cierre debería devolver 0, si no es así significa que ha habido algún problema.*
- *Si bien es importante cerrar todos los archivos que se abrieron, los que resultan críticos son los abiertos para escritura.*
- *Si el programa es interrumpido súbitamente los archivos no se cierran.*

Funciones de manejo

- *La biblioteca estándar dispone de un conjunto de funciones para el manejo de archivos:*

```
fgetc()| getc()     fputc()| putc()
fgets()             fputs()
fprintf()           fscanf()
rewind()            feof()
fseek()             fwrite()
fread()             ungetc()
```

fputc()

- *Se usa para escribir caracteres individuales en una corriente abierta.*

```
int fputc( int ch, FILE *fp );
```

Ej:

```
fputc( 'a', archivo );
```

- *Si la escritura fue exitosa devuelve el mismo valor que escribió, caso contrario retorna EOF.*

fgetc()

- *Se usa para leer caracteres individua-les de una corriente abierta.*

```
int fgetc( FILE *fp );
```

Ej:

```
letra = fgetc( archivo );
```

- *Cuando se alcanza el fin de archivo ob-tenemos EOF.*
- *fgetc y getc son sinónimos.*

fputs()

- *Es el equivalente para archivos del puts() de consola.*

```
char *fputs( char *cadena,
             FILE *fp );
```

Ej:

```
fputs( frase, archivo );
```

- *Si la escritura fue exitosa devuelve la misma cadena que escribió.*

fgets()

- *Es el equivalente para archivos del gets() de consola.*

```
char *fgets( char *cadena, int
                longitud, FILE *fp );
```

Ej:

```
fgets( frase, 80, archivo );
```

- *Devuelve caracteres hasta encontrar nueva línea ó hasta el valor indicado por longitud, lo que ocurra primero.*

fprintf()

- *Para archivos, fprintf() es similar a printf(), permite escribir distintos tipos de variables y constantes, especificando el formato de salida.*

```
int fprintf( FILE *fp,
  const char *cad_contr, argums );
```

Ej:

```
fprintf( archivo, ”%d”, valor );
```

Corrientes estándar

- *Cuando se comienza a ejecutar un programa "C", se abren tres corrientes denominadas estándar porque siempre están, sin necesidad de que se tome acción alguna.*
- *Estas son:*

stdin
stdout
stderr

Corrientes estándar

- ***stdin*** *Significa "standard input" o entrada estándar. Por defecto está asociada al teclado (ese es el archivo).*
- ***stdout*** *Significa "standard output" o salida estándar. Por defecto está asociada a la pantalla.*
- ***stderr*** *Significa "standard error" o salida de diagnóstico. Por defecto está asociada a la pantalla.*

Corrientes estándar

- *Como la entrada por teclado y la salida a pantalla es una operación muy frecuente, se han adoptado formas especiales de la función general fprintf().*
- *Si en:*

  `fprintf( archivo, "%d", valor );`

  reemplazamos **`archivo`** *por* **`stdout`** *escribimos en la salida estándar.*

Corrientes estándar

- *Esto quiere decir que podemos trabajar usando fprintf() ignorando printf(), ya que esta última es un caso especial de fprintf().*
- *Por lo frecuente de la operación, como se dijo antes, se incorporó la función printf() que es simplemente un caso particular de fprintf().*
- *Lo mismo sucede con scanf() y fscanf().*

fscanf()

- *Para archivos, fscanf() es similar a scanf(), permite leer distintos tipos de variables, especificando el formato de entrada.*

```
int fscanf( FILE *fp,
  const char *cad_contr,argums );
```

Ej:

```
fscanf( archivo, ”%d”, &valor );
```

Corrientes estándar

- *La entrada y salida estándar es redirec-cionable desde el sistema operativo.*
- *Por ejemplo si decimos*

  `C:\> generador > sal.gen`

  ó

  `$ generador > sal.gen`

  Estamos redireccionando la salida estándar del programa **`generador`** *al archivo:* **`sal.gen`**

Corrientes estándar

- *La existencia de una salida estándar y otra de diagnóstico, es para dar un camino diferente a los errores.*
- *Esto es especialmente útil en los mecanismos de tuberías (pipes).*

```
prg1 | prg2 | prg3 | prg4 ...
```

la salida estándar que alimenta al proceso siguiente no transmite mensajes de error o advertencias.

Acceso aleatorio

- *Sobre cierta clase de archivos (dispositivos), es posible realizar un acceso aleatorio.*
- *Esto implica moverse hacia atrás y adelante del archivo, para leer o escribir.*
- *Para esto disponemos de las funciones:*

fseek()
rewind()

fseek()

- *Permite posicionar el puntero a una cierta cantidad de caracteres a partir de un punto de origen especificable.*

```
int fseek( FILE *fp, long int
                num_bytes, int origen);
```

- *Ej.:*

```
fseek( archivo, 1000, SEEK_SET );
```

- *El origen depende de dónde se lo computa.*

Origen para fseek()

- *Tenemos tres puntos a partir desde don-de se cuenta el desplazamiento:*

Comienzo de archivo	***SEEK_SET***	***0***
Posición actual	***SEEK_CUR***	***1***
Fin de archivo	***SEEK_END***	***2***

- *Estas constantes están definidas en la biblioteca estándar.*

rewind()

- *En un archivo abierto para lectura/escritura, lleva el puntero al principio del archivo.*

```
void rewind( FILE *fp );
```

- *Ej.:*

```
rewind( archivo );
```

- *Es equivalente a:*

```
fseek( archivo, 0, SEEK_SET );
```

feof()

- *Esta función permite detectar el momento en que se alcanza el fin del archivo.*
- *Esto está representado por la constante **EOF** (**E**nd **O**f **F**ile), definida en la biblioteca estándar.*
- *La forma es:*

```
int feof( FILE *fp );
```

- *Ej:*

```
if( feof( entrada ) ) ...
```

feof()

- *feof() devuelve falso (0) si no se ha alcanzado el fin de archivo y verdadero si el puntero está posicionado en el fin de archivo.*
- *Es especialmente útil cuando estamos recorriendo un archivo mediante un lazo de iteración indeterminado, ej.:*

```
while(!feof(fp)) car = getc(fp);
```

ungetc()

- *Devuelve a una corriente un carácter leido por la función getc() ó fgetc().*
- *La forma es:*

  ```
  int ungetc( int c, FILE *stream );
  ```

  Si la operación resulta exitosa devuelve el carácter ingresado, caso contrario devuelve EOF.
- *Es útil para devolver un centinela.*

Datos Binarios

- *Para transferir un bloque de datos binarios entre la memoria y un archivo o viceversa se utilizan dos funciones:*

fread()
fwrite()

- *No solo sirven para trasladar un grupo de bytes, sino también para escribir datos usando la forma binaria de almacenamiento.*

fread()

- *Lee datos de una corriente.*

```
size_t fread( void *ptr, size_t
  size, size_t n, FILE *stream );
```

ptr *Apunta a un bloque donde se colocan los datos.*

size *Longitud en bytes de cada item leido.*

n *Número de items leídos.*

stream *Apunta a una corriente.*

fread()

- *Ejemplo:*

  `fread( posmem, 10, 4, archivo );`

  El número total de bytes leídos es:

  *total = n * size;*

- *Si la operación fue exitosa, fread() retorna el número de items leídos, caso contrario retorna 0.*

fwrite()

- *Hace la operación inversa de fread(), transfiriendo elementos de la memoria a un almacenamiento secundario.*
- *Uso:*

```
size_t fwrite( const void *ptr,
   size_t size, size_t n,
   FILE *stream );
```

- *El número de bytes escritos es:*

*total = n * size;*

fwrite()

- *Los elementos son:*

ptr *Apunta a un bloque desde donde se obtienen los datos.*

size *Longitud en bytes de cada item escrito.*

n *Número de items escritos.*

stream *Apunta a una corriente.*

Ejemplos Utilización de Archivos

- *A continuación veremos dos programas de ejemplo de utilización de archivos:*

  a) Copia de un archivo

  b) Transferencias en bloque con fread y fwrite.

```c
/* Programa que crea un backup del archivo AUTOEXEC.BAT */
#include <stdio.h>

int main(void)
{
   FILE *in, *out;
   char letra;

   if ((in = fopen("C:\\AUTOEXEC.BAT", "rt")) == NULL){
      printf("No puedo abrir el archivo de entrada.\n");
      return 1;
   }

   if ((out = fopen("C:\\AUTOEXEC.BAK", "wt")) == NULL){
      printf("No puedo abrir el archivo de salida.\n");
      return 1;
   }
   letra = fgetc(in);
   while (!feof(in)) {
      fputc(letra, out);
      letra = fgetc(in);
   }
   fclose(in);
   fclose(out);
   return 0;
}
```

Variante con cadenas

- *Si queremos utilizar funciones de cadenas en lugar de caracteres, deberíamos reemplazar el while por:*

```
while ( fgets(linea, LONGLIN+1, in) != NULL )
  fputs(linea, out);
```

- *Definiendo previamente los siguientes parámetros:*

```
#define LONGLIN 80
int main(void) {
  char linea[LONGLIN];
  ...
```

```
/* Ejemplo con fread() y fwrite() */

#include <string.h>
#include <stdio.h>

int main(void)
{
   FILE *stream;
   char msg[] = "Cadena de prueba ";
   char buf[20];

   if ((stream = fopen("ARCHIVO.TXT", "w+")) == NULL) {
      printf("No puedo abrir archivo de salida.\n");
      return 1;
   }

   /* escribe la cadena inicializada en el archivo */
   if( fwrite(msg, strlen(msg)+1, 1, stream) != 1 ) {
      printf("No puedo escribir en el archivo de salida.\n");
      return 1;
   }

   /* lleva puntero al principio del archivo */
   fseek(stream, SEEK_SET, 0);
```

```
/* Ejemplo con fread() y fwrite(), Continuación */

   /* lee datos del archivo y los muestra */
   if(fread(buf, strlen(msg)+1, 1, stream) != 0)
     printf("%s\n", buf);
   else {
     printf( "Error en la lectura del archivo\n" );
     return 1;
   }

   fclose(stream);
   return 0;
}
```

Argumentos de línea de comando

```
#include <stdio.h>
int main( int argc, char *argv[] ) {
  int i;
  printf( "La cantidad de argumentos es: %d\n", argc );
  printf( "El nombre del programa es: %s\n", argv[0] );
  for( i=1; i<argc; i++ )
    printf( "El argumento %2d es: %s\n", i, argv[i] );
  return 0;
}
```